

May 29th Weekly Report

Erik Bigwood

July 26, 2025

Abstract

For this report, I will discuss some progress on learning the double pendulum system via the a naive (non-Lagrangian) neural network implementation. Given larger neural network sizes and training run times, this method has converged to results that can be quite accurate, and at worst capture the qualitative behavior of the system, but appears to lack the physical constraints (energy conservation, etc.) that make the learned Lagrangian method so powerful.

Contents

1	Neural Net Structure	2
2	Neural Network Results	3
2.1	Basic architecture	3
2.2	Results	4
A	Bibliography	7

1 Neural Net Structure

Basic neural nets I'll start by providing a quick summary of neural net structures. In this work I'll be restricting to 'multi-level perceptron' (MLP) networks[Yu2021]. These are the common feedforward-type networks with fully connected layers. More specifically, these are maps from an input space $\mathbb{R}^m$ to an output space $\mathbb{R}^n$ with l internal layers of N_k nodes per 'hidden' layer. Linking each layer is an array of weights $\sigma_{k,k+1}$, vector of biases $b_{k,k+1}$, and a non-linear activation function $f_{k,k+1}$. Each layer's state is contained in a vector of 'activations' x_k .

Evaluation of this neural network (NN) comes in three stages. To begin, propagate the input into the first hidden layer:

$$x_1 = f_{0,1}(\sigma_{0,1}x_0 + b_{0,1})$$

Where x_0 is the input to the NN. For the remaining hidden layers, iterate the same procedure:

$$x_{k+1} = f_{k,k+1}(\sigma_{k,k+1}x_k + b_{k,k+1})$$

For the final output, I apply the weights and biases but not the non-linear activation function to the activation of the final hidden layer x_l

$$x_{final} = \sigma_{l,final}x_l + b_{l,final}$$

For the remainder of this report, I'll stick to using the same non-linear activation function for all layers. In particular, I've chosen the rectified linear unit (ReLU) function, whose form is:

$$ReLU(x) = x \cdot (x > 0)$$

This is 0 for any $x \leq 0$ and x for any $x > 0$. ReLU is used very widely within industry as it doesn't suffer from the 'vanishing gradient' problem of other activation functions like the sigmoid, tanh, etc. functions, but can produce 'dead' neurons where the input to a neuron is always < 0 . These situations produce neurons that effectively contribute nothing to the results and can be pruned away, but generally waste computational resources.

Compared to the exact Lagrangian, these neural nets often trade massively increased consumption of computational resources for expressivity. For a MLP mapping $\mathbb{R}^m \rightarrow \mathbb{R}^n$ with l hidden layers and N_k neurons per layer, the number of independent parameters Ω is:

$$\Omega = [(m+1)N_1] \left[\sum_{k=1}^{l-1} (N_k+1)N_{k+1} \right] [(n+1)N_l]$$

Assuming the same number of nodes at each layer, this becomes

$$\Omega = (m+1)(n+1)lN^2$$

Which can quickly hit the limits of whatever hardware is used.

Loss functions Expanding to NNs also requires some restructuring of the training loss function. For smaller numbers of parameters and more inherent structure in the parameter meanings (i.e. each parameter corresponding to a term like $q_0^i q_1^j$), the general shape of the loss landscape should be relatively simple. With a NN, there are many more parameters (see above), and they do not correspond to anything concrete in the internal structure – they are simply numbers in a black box. In the rest of this report, I will be using the mean-squared-error (L^2) loss function provided by PyTorch.

2 Neural Network Results

2.1 Basic architecture

Model architecture The model under consideration here is a relatively simple feed-forward network. I’ll restrict my discussion to the most successful model found – a sequential NN with 5 hidden layers of 512 nodes each, using the ReLU activation function. I take the angular positions $(q_0, q_1) \bmod 2\pi$ due to the underlying symmetry in angle space. In total, this becomes a non-linear map from phase space to accelerations $\vec{x} \in \mathbb{R}^4 \rightarrow \vec{y} \in \mathbb{R}^2$:

$$\begin{aligned} i\vec{n} &= \vec{x} \bmod 2\pi \\ \vec{l}_1 &= \text{ReLU} \left[\sigma_{in,l_1} i\vec{n} + b_{in,l_1} \right] \\ \vec{l}_2 &= \text{ReLU} \left[\sigma_{l_1,l_2} \vec{l}_1 + b_{l_1,l_2} \right] \\ \vec{l}_3 &= \text{ReLU} \left[\sigma_{l_2,l_3} \vec{l}_2 + b_{l_2,l_3} \right] \\ \vec{l}_4 &= \text{ReLU} \left[\sigma_{l_3,l_4} \vec{l}_3 + b_{l_3,l_4} \right] \\ \vec{l}_5 &= \text{ReLU} \left[\sigma_{l_4,l_5} \vec{l}_4 + b_{l_4,l_5} \right] \\ \vec{y} &= \sigma_{l_5,out} \vec{l}_5 + b_{l_5,out} \end{aligned}$$

Where $\sigma_{layer_i, layer_{i+1}}$ are the weights mapping layer i to layer $i + 1$, and $b_{layer_i, layer_{i+1}}$ are the biases associated to the same transition map. These parameters are float32, so this model consumes

$$(4 \text{ bytes}) \cdot \left[(4 \cdot 512) + 5(512 \cdot 512) + 5(512) \right] \approx 1.3 \text{ Mbyte} \quad (1)$$

in memory. Compared to industry models, this is extremely small, but optimizing this model still requires substantial amounts of compute time. I’ve tried to transfer this model to both my Apple GPU and Google Colab’s T4 GPUs, but these systems fail to produce significantly better performance than my laptop’s CPU, likely due to memory bandwidth limitations and GPU overhead. Given the small size of this model, the scaling of these GPU systems is not enough to overcome the overhead, but this would likely change with a much larger model or improvements to the model architecture or datatype. Modern GPUs are not generally optimized to run FP32 calculations, and instead focus on reduced precision formats like FP16 or BF16 (or smaller!).

To reduce memory management bottlenecks, I’ve incorporated PyTorch’s TensorDataset and DataLoader functionality. These manage samples and batching during the training loop,

hiding some multi-threading capabilities to better utilize the CPU/GPU hardware without requiring developers to design and implement that functionality.

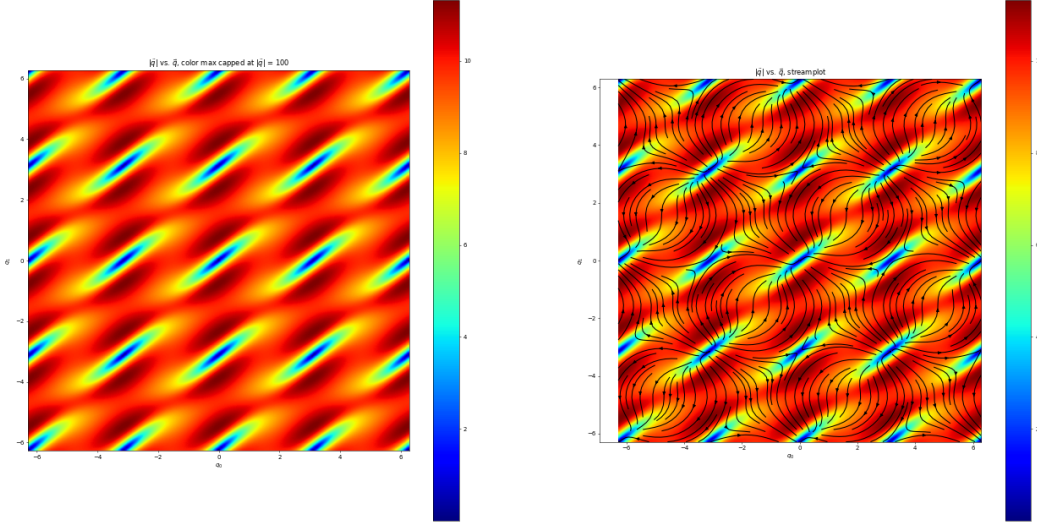
Algorithm The algorithm is as follows:

- Generate a synthetic dataset consisting of points $(q_0, q_1, \dot{q}_0, \dot{q}_1, \ddot{q}_0, \ddot{q}_1)$ by randomly sampling a bounded region in $(q_0, q_1, \dot{q}_0, \dot{q}_1)$ space and calculating true accelerations. Create a TensorDataset object containing these samples, configured to provide a set batch size and shuffle at each iteration.
- Construct a PyTorch neural network class (containing a method for forward propagation) and instantiate it. Define a loss function and optimizer method.
- Call the training loop method and monitor for convergence.
- Analyze the resulting network and its performance and accuracy.
- Modify the training dataset, learning rate, optimizer, etc. to continue to improve results

2.2 Results

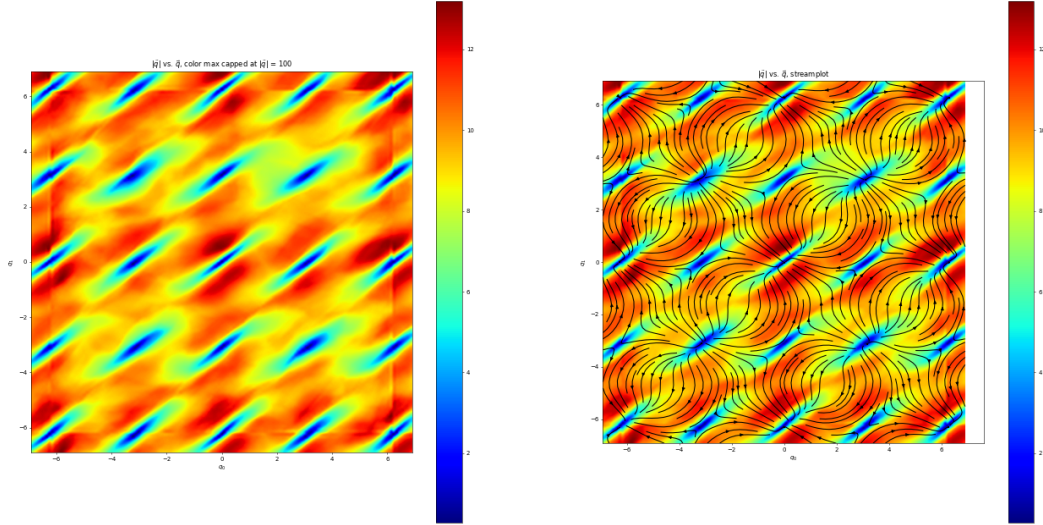
For ease of visualization, consider a slice through position space with zero velocities and plot the magnitude of the accelerations and a streamplot:

Figure 1: True acceleration vs. position, zero velocity



After several hours of training, the model results (for the zero-velocity slice) match the true results relatively well:

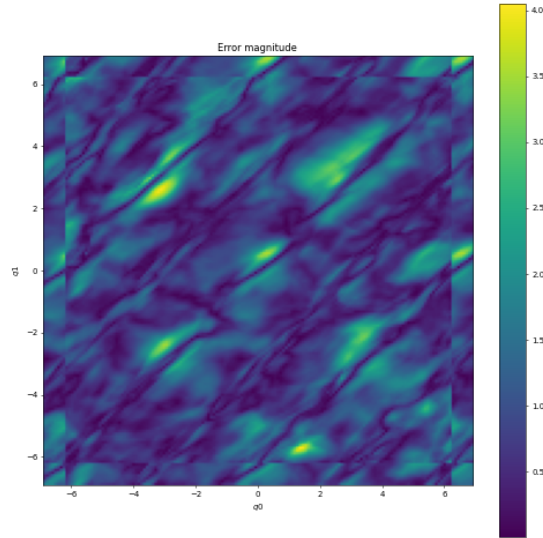
Figure 2: Model acceleration vs. position, zero velocity



The double pendulum equilibria (zero acceleration) at $q = 0 \bmod \pi$ are all present and well-defined, and the areas immediately surrounding each equilibrium show the correct stream behavior. However, some of the intermediate areas are not well-represented by the model and carry a deformation from the true system.

Now plotting the magnitude of the error in acceleration, we see that the model generally fits quite well but has a couple of regions with large error. Additionally, the boundary between one domain (e.g. $\{q \in (-2\pi, 2\pi)\} \rightarrow \{q \in (2\pi, 4\pi)\}$) shows a discontinuity coming from taking the positions mod 2π .

Figure 3: Model acceleration vs. position, zero velocity



While animating the results, I noticed that the resulting model trajectories often develop a position offset relative to the true trajectories, but tend to match qualitative behavior (spinning quickly, looping, transferring energy from one pendulum to the other, etc.) at the same time as the true system.

Appendix A Bibliography